Android Studio — Java + XML

Code Snippets Notes: Fragments, Dynamic UI & Menus

Practical reference for MCA / Android Studio laboratory work

- Fragments and Fragment lifecycle
- Fragment XML and Java
- Adding, replacing and removing fragments
- Fragment navigation and passing data
- Dynamic UI creation entirely in Java
- Dynamic TextView, EditText, Button and layouts
- Dynamic lists and event handling
- Options menu, menu XML and Java callbacks
- Context menu
- Popup menu
- Menu items, icons, groups and visibility
- Integrated mini examples and quick revision

Java + XML • Reusable snippets • Lab-oriented

Contents

1. Fragments — fundamentals and lifecycle
2. Creating a Fragment
3. Adding and replacing Fragments
4. Fragment communication and data
5. Fragment lifecycle reference
6. Dynamic UI in Java
7. Dynamic layouts and widgets
8. Dynamic event handling and form generation
9. Dynamic lists
10. Android Menus — overview
11. Options Menu
12. Context Menu
13. Popup Menu
14. Menu item handling and advanced patterns
15. Integrated lab examples
16. Quick revision sheet

> Convention: Examples use AndroidX and Java. XML belongs in res/layout or res/menu as indicated.

1. Fragments — Fundamentals

A Fragment is a reusable portion of an Activity's UI and behavior. A Fragment has its own lifecycle, can have its own XML layout, and is hosted by an Activity. Use AndroidX androidx.fragment.app.Fragment in modern projects.

Basic Fragment class

```
public class HomeFragment extends Fragment {

    public HomeFragment() {
        super(R.layout.fragment_home);
    }

    @Override
    public void onViewCreated(
            View view, Bundle savedInstanceState) {
        super.onViewCreated(view, savedInstanceState);

        TextView tv = view.findViewById(R.id.tvTitle);
        tv.setText("Home Fragment");
    }
}
```

fragment_home.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:gravity="center"
    android:padding="16dp">

    <TextView
        android:id="@+id/tvTitle"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Home"
        android:textSize="24sp" />

    <Button
        android:id="@+id/btnNext"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Next" />
</LinearLayout>
```

2. Creating a Fragment Host

The Activity normally provides a container into which fragments are inserted or replaced. FragmentContainerView is a convenient AndroidX container.

Activity XML — FragmentContainerView

```
<androidx.fragment.app.FragmentContainerView
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:id="@+id/fragmentContainer"
    android:layout_width="match_parent"
    android:layout_height="match_parent" />
```

Add the first Fragment

```
getSupportFragmentManager()
    .beginTransaction()
    .add(R.id.fragmentContainer, new HomeFragment())
    .commit();
```

Use replace() when changing the contents of the container. Use add() when you intentionally want to add another fragment without removing the current one.

3. Add, Replace and Remove Fragments

Replace Fragment

```
getSupportFragmentManager()
    .beginTransaction()
    .replace(
        R.id.fragmentContainer,
        new ProfileFragment()
    )
    .commit();
```

Replace + Back Stack

```
getSupportFragmentManager()
    .beginTransaction()
    .replace(
        R.id.fragmentContainer,
        new ProfileFragment()
    )
    .addToBackStack(null)
    .commit();
```

Remove Fragment

```
Fragment fragment = getSupportFragmentManager()
        .findFragmentByTag("HOME");

if (fragment != null) {
    getSupportFragmentManager()
        .beginTransaction()
        .remove(fragment)
        .commit();
}
```

Give a Fragment a tag

```
getSupportFragmentManager()
    .beginTransaction()
    .replace(
        R.id.fragmentContainer,
        new HomeFragment(),
        "HOME"
    )
    .commit();
```

4. Passing Data to a Fragment

A Bundle is a simple way to pass initial data when creating a Fragment.

Activity → Fragment

```
Bundle bundle = new Bundle();
bundle.putString("username", "Deepak");
bundle.putInt("age", 25);

ProfileFragment fragment = new ProfileFragment();
fragment.setArguments(bundle);

getSupportFragmentManager()
    .beginTransaction()
    .replace(R.id.fragmentContainer, fragment)
    .commit();
```

Read Bundle inside Fragment

```
@Override
public void onViewCreated(
        View view, Bundle savedInstanceState) {
    super.onViewCreated(view, savedInstanceState);

    Bundle args = getArguments();

    if (args != null) {
        String username = args.getString("username");
        int age = args.getInt("age");

        TextView tv = view.findViewById(R.id.tvInfo);
        tv.setText(username + " - " + age);
    }
}
```

5. Fragment → Activity Communication

For simple lab programs, an interface callback is a useful way to send an event from a Fragment to its host Activity.

Interface callback pattern

```
public class HomeFragment extends Fragment {

    interface OnNameSelected {
        void onNameSelected(String name);
    }

    private OnNameSelected listener;

    @Override
```

```
    public void onAttach(Context context) {
        super.onAttach(context);

        if (context instanceof OnNameSelected) {
            listener = (OnNameSelected) context;
        }
    }

    private void selectName() {
        if (listener != null) {
            listener.onNameSelected("Deepak");
        }
    }
}
```

Activity implements callback

```
public class MainActivity extends AppCompatActivity
        implements HomeFragment.OnNameSelected {

    @Override
    public void onNameSelected(String name) {
        Toast.makeText(
            this,
            "Selected: " + name,
            Toast.LENGTH_SHORT
        ).show();
    }
}
```

6. Fragment Lifecycle

Callback	Typical purpose
onAttach()	Fragment is attached to its host Activity/context.
onCreate()	Initialize non-UI state.
onCreateView()	Inflate/create the Fragment's UI.
onViewCreated()	Access views and attach listeners.
onStart()	Fragment becomes visible.
onResume()	Fragment is active and interactive.
onPause()	No longer in the foreground.
onStop()	No longer visible.
onDestroyView()	Fragment's view hierarchy is destroyed.
onDestroy()	Fragment object is being destroyed.
onDetach()	Fragment is detached from Activity.

For view-related references, onViewCreated() is a convenient place to call view.findViewById(). The view hierarchy may be destroyed while the Fragment object remains.

7. Dynamic UI in Java

Dynamic UI means creating, modifying or removing Views at runtime using Java instead of defining every View in XML. The basic pattern is: create View → configure LayoutParams → add it to a parent → attach listener.

Create a TextView dynamically

```
LinearLayout layout = findViewById(R.id.mainLayout);

TextView tv = new TextView(this);

tv.setText("Created in Java");
tv.setTextSize(20);
tv.setPadding(16, 16, 16, 16);

layout.addView(tv);
```

Create a Button dynamically

```
Button button = new Button(this);
```

```
button.setText("Click Me");

button.setOnClickListener(v -> {
    Toast.makeText(
        this,
        "Dynamic button clicked",
        Toast.LENGTH_SHORT
    ).show();
});

layout.addView(button);
```

Create an EditText dynamically

```
EditText editText = new EditText(this);

editText.setHint("Enter your name");
editText.setInputType(
    InputType.TYPE_CLASS_TEXT
);

layout.addView(editText);
```

8. Dynamic Layouts and LayoutParams

Every View added to a ViewGroup needs suitable LayoutParams. The type of LayoutParams must match the parent layout.

Dynamic LinearLayout

```
LinearLayout dynamicLayout =
        new LinearLayout(this);

dynamicLayout.setOrientation(
        LinearLayout.VERTICAL);

dynamicLayout.setPadding(16, 16, 16, 16);

setContentView(dynamicLayout);
```

LayoutParams for match_parent / wrap_content

```
LinearLayout.LayoutParams params =
        new LinearLayout.LayoutParams(
            ViewGroup.LayoutParams.MATCH_PARENT,
            ViewGroup.LayoutParams.WRAP_CONTENT
        );

TextView tv = new TextView(this);
tv.setText("Dynamic Text");
tv.setLayoutParams(params);

dynamicLayout.addView(tv);
```

Dynamic width using weight

```
LinearLayout.LayoutParams params =
        new LinearLayout.LayoutParams(
            0,
            ViewGroup.LayoutParams.WRAP_CONTENT,
            1f
        );

Button button = new Button(this);
button.setText("Equal Width");
button.setLayoutParams(params);

rowLayout.addView(button);
```

9. Dynamic UI — Add a Form Field on Button Click

activity_main.xml

```
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:id="@+id/mainLayout"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="16dp">

    <Button
        android:id="@+id/btnAdd"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Add Field" />
```

```
    <LinearLayout
        android:id="@+id/fieldContainer"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical" />
</LinearLayout>
```

MainActivity.java

```
LinearLayout fieldContainer =
        findViewById(R.id.fieldContainer);

Button btnAdd = findViewById(R.id.btnAdd);

btnAdd.setOnClickListener(v -> {

    EditText field = new EditText(this);

    int number = fieldContainer.getChildCount() + 1;

    field.setHint("Field " + number);

    LinearLayout.LayoutParams params =
        new LinearLayout.LayoutParams(
            ViewGroup.LayoutParams.MATCH_PARENT,
            ViewGroup.LayoutParams.WRAP_CONTENT
        );

    field.setLayoutParams(params);

    fieldContainer.addView(field);
});
```

10. Dynamic UI — Remove a View

Remove one View

```
fieldContainer.removeView(field);
```

Remove all dynamically added Views

```
fieldContainer.removeAllViews();
```

Access a dynamically created child

```
int count = fieldContainer.getChildCount();

for (int i = 0; i < count; i++) {
    View child = fieldContainer.getChildAt(i);

    if (child instanceof EditText) {
        String value = ((EditText) child)
                .getText()
                .toString();
    }
}
```

11. Dynamic UI — Programmatically Create a Row

Two buttons in one row

```
LinearLayout row = new LinearLayout(this);
row.setOrientation(LinearLayout.HORIZONTAL);

Button yes = new Button(this);
yes.setText("YES");

Button no = new Button(this);
no.setText("NO");

LinearLayout.LayoutParams p =
    new LinearLayout.LayoutParams(
        0,
        ViewGroup.LayoutParams.WRAP_CONTENT,
        1f
    );

row.addView(yes, p);
row.addView(no, p);

mainLayout.addView(row);
```

12. Dynamic UI — Create Views from an Array

Generate a list of TextViews

```
String[] subjects = {
    "Java", "Android", "Database", "Python"
};

for (String subject : subjects) {

    TextView tv = new TextView(this);

    tv.setText(subject);
    tv.setTextSize(18);
    tv.setPadding(12, 12, 12, 12);

    tv.setOnClickListener(v -> {
        TextView clicked = (TextView) v;

        Toast.makeText(
            this,
            clicked.getText(),
            Toast.LENGTH_SHORT
        ).show();
    });

    container.addView(tv);
}
```

13. Menus in Android — Overview

The three important menu patterns for traditional Android View applications are Options Menu, Context Menu and Popup Menu. Menu XML is normally stored in res/menu/.

Typical menu XML structure

```
<?xml version="1.0" encoding="utf-8"?>
<menu xmlns:android="http://schemas.android.com/apk/res/android">

    <item
        android:id="@+id/menu_save"
        android:title="Save" />

    <item
        android:id="@+id/menu_settings"
        android:title="Settings" />

</menu>
```

14. Options Menu

The Options Menu provides actions for an Activity. Depending on the Android version, theme and device, menu items may appear in the app bar or overflow menu.

res/menu/main_menu.xml

```
<?xml version="1.0" encoding="utf-8"?>
<menu xmlns:android="http://schemas.android.com/apk/res/android">

    <item
        android:id="@+id/action_add"
        android:title="Add" />

    <item
        android:id="@+id/action_search"
        android:title="Search" />

    <item
        android:id="@+id/action_settings"
        android:title="Settings" />
</menu>
```

Inflate Options Menu in Activity

```
@Override
public boolean onCreateOptionsMenu(Menu menu) {

    getMenuInflater().inflate(
        R.menu.main_menu,
        menu
    );

    return true;
}
```

Handle menu item click

```
@Override
public boolean onOptionsItemSelected(
```

```
        @NonNull MenuItem item) {

    int id = item.getItemId();

    if (id == R.id.action_add) {
        Toast.makeText(
            this, "Add clicked",
            Toast.LENGTH_SHORT
        ).show();
        return true;
    }

    if (id == R.id.action_settings) {
        // Open settings
        return true;
    }

    return super.onOptionsItemSelected(item);
}
```

15. Menu Icons and showAsAction

To display an icon, add a drawable resource and reference it from the menu item. showAsAction controls whether an action can appear directly in the app bar.

Menu item with icon

```
<item
    android:id="@+id/action_save"
    android:title="Save"
    android:icon="@drawable/ic_save"
    android:showAsAction="ifRoom" />
```

Common values: never keeps the item in overflow; ifRoom shows it directly when there is room; always requests direct display. Exact visual behavior depends on the app bar/theme/device.

16. Menu Groups and Checkable Items

Grouped menu items

```
<group
    android:id="@+id/group_edit"
    android:checkableBehavior="single">

    <item
        android:id="@+id/action_normal"
        android:title="Normal" />

    <item
        android:id="@+id/action_compact"
        android:title="Compact" />

</group>
```

Checkable menu item

```
<item
    android:id="@+id/action_dark"
    android:title="Dark Mode"
    android:checkable="true" />
```

17. Dynamically Change Menu Items

Use onPrepareOptionsMenu() when the menu needs to reflect the current state of the Activity.

Show/hide an item

```
@Override
public boolean onPrepareOptionsMenu(Menu menu) {

    MenuItem save =
        menu.findItem(R.id.action_save);

    save.setVisible(isFormValid);

    return super.onPrepareOptionsMenu(menu);
}
```

Refresh the menu

```
invalidateOptionsMenu();
```

18. Context Menu

A Context Menu is associated with a particular View, commonly appearing after a long press.

Register a View for Context Menu

```
TextView textView = findViewById(R.id.tvName);

registerForContextMenu(textView);
```

Create Context Menu

```
@Override
public void onCreateContextMenu(
        ContextMenu menu,
        View v,
        ContextMenu.ContextMenuInfo menuInfo) {

    super.onCreateContextMenu(menu, v, menuInfo);

    menu.setHeaderTitle("Choose Action");

    menu.add(0, 1, 0, "Edit");
    menu.add(0, 2, 1, "Delete");
}
```

Handle Context Menu selection

```
@Override
public boolean onContextItemSelected(
        @NonNull MenuItem item) {

    if (item.getItemId() == 1) {
        // Edit
        return true;
    }

    if (item.getItemId() == 2) {
        // Delete
        return true;
    }

    return super.onContextItemSelected(item);
}
```

19. Popup Menu

A PopupMenu is attached to a particular View and is useful for small contextual actions.

Popup menu XML — res/menu/popup_menu.xml

```
<menu xmlns:android="http://schemas.android.com/apk/res/android">

    <item
        android:id="@+id/action_edit"
        android:title="Edit" />

    <item
        android:id="@+id/action_delete"
        android:title="Delete" />

</menu>
```

Show PopupMenu from a Button

```
Button button = findViewById(R.id.btnMore);

button.setOnClickListener(v -> {

    PopupMenu popup =
        new PopupMenu(MainActivity.this, v);

    popup.getMenuInflater().inflate(
        R.menu.popup_menu,
        popup.getMenu()
    );

    popup.setOnMenuItemClickListener(item -> {

        if (item.getItemId() == R.id.action_edit) {
            Toast.makeText(
                this, "Edit",
                Toast.LENGTH_SHORT
            ).show();
            return true;
        }
```

```
        if (item.getItemId() == R.id.action_delete) {
            Toast.makeText(
                this, "Delete",
                Toast.LENGTH_SHORT
            ).show();
            return true;
        }

        return false;
    });

    popup.show();
});
```

20. Options Menu vs Context Menu vs Popup Menu

Menu	Typical use	How it appears
Options Menu	Activity-level actions/settings	App bar / overflow
Context Menu	Actions for a selected View	Usually long press
Popup Menu	Small contextual action list	Anchored to a View

21. Integrated Lab — Fragment Navigation

Activity XML

```
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="horizontal">

        <Button
            android:id="@+id/btnHome"
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:layout_weight="1"
            android:text="Home" />

        <Button
            android:id="@+id/btnProfile"
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:layout_weight="1"
            android:text="Profile" />
    </LinearLayout>

    <androidx.fragment.app.FragmentContainerView
        android:id="@+id/container"
        android:layout_width="match_parent"
        android:layout_height="0dp"
        android:layout_weight="1" />
</LinearLayout>
```

MainActivity.java

```
Button home = findViewById(R.id.btnHome);
Button profile = findViewById(R.id.btnProfile);

home.setOnClickListener(v ->
    getSupportFragmentManager()
        .beginTransaction()
        .replace(R.id.container, new HomeFragment())
        .commit()
);

profile.setOnClickListener(v ->
    getSupportFragmentManager()
        .beginTransaction()
        .replace(R.id.container, new ProfileFragment())
        .addToBackStack(null)
        .commit()
);
```

22. Integrated Lab — Dynamic Form

Create fields and collect their values

```
LinearLayout container =
        findViewById(R.id.fieldContainer);

Button add = findViewById(R.id.btnAdd);
Button submit = findViewById(R.id.btnSubmit);

add.setOnClickListener(v -> {

    EditText input = new EditText(this);
    input.setHint("Enter value");

    container.addView(input);
});

submit.setOnClickListener(v -> {

    for (int i = 0;
         i < container.getChildCount();
         i++) {

        View child = container.getChildAt(i);

        if (child instanceof EditText) {
            String value = ((EditText) child)
                    .getText()
                    .toString()
                    .trim();

            Log.d("FORM", value);
        }
    }
});
```

23. Integrated Lab — Options Menu + Fragment

Menu XML

```
<menu xmlns:android="http://schemas.android.com/apk/res/android">

    <item
        android:id="@+id/action_home"
        android:title="Home" />

    <item
        android:id="@+id/action_profile"
        android:title="Profile" />

    <item
        android:id="@+id/action_exit"
        android:title="Exit" />
</menu>
```

Java

```
@Override
public boolean onCreateOptionsMenu(Menu menu) {
    getMenuInflater().inflate(
        R.menu.main_menu, menu
    );
    return true;
}

@Override
public boolean onOptionsItemSelected(
        @NonNull MenuItem item) {

    int id = item.getItemId();

    if (id == R.id.action_home) {
        openHome();
        return true;
    }

    if (id == R.id.action_profile) {
        openProfile();
        return true;
    }

    if (id == R.id.action_exit) {
        finish();
        return true;
    }
```

```
    return super.onOptionsItemSelected(item);
}
```

24. Quick Revision Sheet

Task	Essential syntax
Find Fragment	getSupportFragmentManager().findFragmentByTag("TAG")
Add Fragment	beginTransaction().add(R.id.container, new HomeFragment()).commit();
Replace Fragment	beginTransaction().replace(R.id.container, new HomeFragment()).commit();
Back stack	.addToBackStack(null)
Fragment argument	fragment.setArguments(bundle)
Read argument	getArguments()
Create View	new TextView(this), new Button(this), new EditText(this)
Add View	parentLayout.addView(view)
Remove View	parentLayout.removeView(view)
Remove all	parentLayout.removeAllViews()
Child count	parentLayout.getChildCount()
Get child	parentLayout.getChildAt(index)
Options Menu	onCreateOptionsMenu(Menu menu)
Menu click	onOptionsItemSelected(MenuItem item)
Refresh menu	invalidateOptionsMenu()
Context Menu	registerForContextMenu(view)
Popup Menu	new PopupMenu(this, anchorView)

25. Common Imports

Frequently used imports

```
import android.content.Context;
import android.os.Bundle;
import android.text.InputType;
import android.util.Log;
import android.view.ContextMenu;
import android.view.Menu;
import android.view.MenuItem;
import android.view.View;
import android.view.ViewGroup;
import android.widget.Button;
import android.widget.EditText;
import android.widget.LinearLayout;
import android.widget.PopupMenu;
import android.widget.TextView;
import android.widget.Toast;

import androidx.annotation.NonNull;
import androidx.appcompat.app.AppCompatActivity;
import androidx.fragment.app.Fragment;
```

26. Practical Lab Checklist

- Create a Fragment XML layout and corresponding Java Fragment class.
- Use FragmentContainerView in an Activity and replace fragments using FragmentManager.
- Pass a String/int through Bundle arguments and display it in a Fragment.
- Create TextView, EditText and Button dynamically and add them to a LinearLayout.
- Create a dynamic form where the user can add and remove fields.

- Loop through child Views and collect EditText values.
- Create an Options Menu using res/menu XML and handle clicks in Java.
- Create a Context Menu for a TextView or ListView item.
- Create a PopupMenu anchored to a Button.
- Use invalidateOptionsMenu() when menu visibility/state must change.
- Test fragment back-stack behavior using the device Back button.

End of notes — Fragments • Dynamic UI • Menus